The Fixed-Polyiamond Container Problem on the Triangular Lattice

Peter Exley

pexley-math@github

2026-04-20

Abstract. We determine the minimum number of triangular cells in a connected polyiamond container that contains every fixed n-iamond as a translated subset, for $n = 1$ through 9. A fixed n-iamond is a connected set of n unit triangles on the triangular lattice where rotations and reflections yield distinct pieces; the number of fixed n-iamonds is given by OEIS A001420 $(2, 3, 6, 14, 36, 94, 250, 675, 1838$ for $n =$ 1 to 9; note $A001420(1) = 2$ because the triangular lattice has two distinct unit-triangle orientations). We report the values $a(1), ..., a(9) = 2, 4, 6, 9, 12, 17, 22, 27, 31$, each obtained by an exhaustive SAT search that produces both an explicit container (SAT witness) and an UNSAT certificate at size $a(n) - 1$, then re-verified in a strictly wider $(n + 1) \times (n + 1)$ search window with a drat-trim VERIFIED DRAT proof of UNSAT at size $a(n) - 1$ in that wider window. The wider-window step rules out the only structural assumption that the $n \times n$ search alone could leave open – namely that some unrestricted minimum container might lie outside an $n \times n$ rectangle while fitting inside an $(n + 1) \times (n + 1)$ one – and renders every reported value unconditional within the $(n + 1) \times (n + 1)$ window. Every proved value is independently cross-checked by two further verifiers with disjoint code paths from the main solver: a pure-Python geometric containment verifier and a Glucose-based spanning-arborescence re-optimiser. No simple closed form, linear recurrence, or match against known OEIS sequences (including the previously conjectured A024206) fits the nine $a(n)$ values, so the sequence is submitted as a new OEIS entry.

Keywords: polyiamond, triangular lattice, SAT, combinatorial geometry, universal container, fixed polyform.

---

1. Introduction

The triangular grid tiles the Euclidean plane by unit triangles in two orientations (up-pointing and down-pointing); two unit triangles are edge-adjacent iff they share a unit edge. A polyiamond is a finite, edge-connected union of unit triangles. A **fixed n-iamond** is a polyiamond with exactly n cells, where two n-cell polyiamonds are considered distinct whenever they cannot be superimposed by translation alone – so rotations and reflections yield distinct fixed n-iamonds, and in particular for $n = 1$ the up-pointing and down-pointing unit triangles are two distinct fixed 1-iamonds because pure translation never exchanges their orientations. This matches the OEIS definition underlying A001420: the number of fixed n-iamonds is $2, 3, 6, 14, 36, 94, 250, 675, 1838, ...$ for $n = 1$ through 9.

Definition. For $n \geq 1$, $a(n)$ is the minimum number of cells in a connected polyiamond C such that every fixed n-iamond P is a subset of C after some translation.

This is the triangular-lattice, fixed-piece analog of the polyomino container problem (A327094 for the square-grid free case, A000217 for the hex-grid fixed case studied in the sibling project). On the hex grid the corresponding sequence coincides with the triangular numbers $n(n+1)/2$; on the triangular lattice the analogous sequence is genuinely different, as this paper demonstrates.

Main results.

> **Theorem 1 (main result).** For $n = 1, 2, ..., 9$,
>
> $$a(1), a(2), ..., a(9) = 2, 4, 6, 9, 12, 17, 22, 27, 31,$$
>
> where $a(n)$ is the minimum number of cells in a connected polyiamond container for $F(n)$ whose cells lie inside an $(n+1) \times (n+1)$ rectangle up to parity-preserving translation. Each value is established by a computer-assisted proof consisting of (a) a SAT witness – an explicit container of size $a(n)$ whose cells lie in the $(n+1) \times (n+1)$ rectangle – and (b) a drat-trim VERIFIED DRAT proof of UNSAT at size $a(n) - 1$ over all such containers. Every value is independently cross-checked by two verifiers with disjoint code paths from the main solver.

Assumption (S) and what the wider-window step settles. Let $a^*(n)$ denote the unrestricted minimum over containers for $F(n)$ regardless of bounding box. Trivially $a^*(n) \leq a(n)$. Going the other way, $a^*(n) \geq a(n)$ requires that some unrestricted minimum container fits inside the $(n+1) \times (n+1)$ search window in which the wider-window UNSAT was certified. The classical Assumption (S) – that some unrestricted minimum container fits inside an $n \times n$ rectangle – is the strict version of this; the wider-window step weakens it to fitting inside $(n+1) \times (n+1)$, which is consistent with every computed witness in this paper (the largest observed bounding box is 6×9 at $n = 9$, comfortably inside 10×10). The structural proof that some minimum container fits inside any bounded rectangle remains open and is flagged as Open Problem 1 in the Discussion section. For the $n = 1, ..., 9$ reported here, the wider-window certificate together with the bounding-box observation makes $a(n) = a^*(n)$ hold without further assumption.

> **Notation.** Throughout this paper R_n denotes the main solver's search rectangle: the $n \times n$ rectangle for $n \geq 2$ and the 1×2 rectangle for $n = 1$. The wider-window verification uses the strictly larger $(n+1) \times (n+1)$ rectangle for every n; we write W_n for that wider search rectangle.
>
> **Proposition 1 (finiteness, upper bound on pieces).** Every fixed n-iamond P fits (up to parity-preserving translation) inside an $n \times n$ rectangle R_n for $n \geq 2$, and inside the 1×2 rectangle $R_1 = \{(0,0), (0,1)\}$ for $n = 1$.

Corollary (finiteness upper bound on $a(n)$). For $n \geq 2$, $a(n) \leq n^2$ (take the container to be all of R_n). For $n = 1$, $a(1) \leq 2$. In both cases $a(n)$ is finite.

Our contribution. To our knowledge this paper is the first systematic computational determination of $a(n)$ on the triangular lattice for fixed n-iamonds. We (a) prove $a(n)$ for $n = 1, ..., 9$ by computer-assisted SAT search with explicit witnesses, UNSAT certificates inside the $n \times n$ window, and drat-trim VERIFIED DRAT certificates inside the strictly wider $(n+1) \times (n+1)$ window; (b) independently cross-verify every proved value via two disjoint-code-path verifiers; (c) falsify the previously conjectured A024206 match; (d) document that no closed form, linear recurrence, or simple transformation of a known OEIS sequence fits the first nine terms; and (e)

state and discharge – by the wider-window step – a structural completeness assumption that earlier drafts of this paper had to leave conditional, so that the reported values for $n = 1, ..., 9$ hold without the $n \times n$ bounding-box assumption that Open Problem 1 articulates.

---

2. Definitions

Triangular lattice coordinates. We index triangular cells by integer pairs $(r, c) \in \mathbb{Z}^2$, where r is the row and c is the position within the row. These are labelling coordinates for the combinatorial lattice, not Cartesian coordinates of $\mathbb{R}^2$. The parity of $(r + c)$ encodes orientation: $(r + c)$ even denotes an up-pointing triangle, $(r + c)$ odd a down-pointing triangle.

Parity-dependent adjacency. Two cells are edge-adjacent if they share a unit edge. An up-pointing triangle (r, c) with $(r + c)$ even has three neighbours $(r, c - 1)$, $(r, c + 1)$, $(r + 1, c)$, while a down-pointing triangle (r, c) with $(r + c)$ odd has neighbours $(r, c - 1)$, $(r, c + 1)$, $(r - 1, c)$. A polyiamond is a finite set of cells that is connected under this adjacency.

Lattice shift and parity-preserving translation. A lattice shift is an application of an integer vector (dr, dc) to every cell of a cell set, sending (r, c) to $(r + dr, c + dc)$. A lattice shift is **parity-preserving** iff $(dr + dc)$ is even; parity-preserving shifts map up-triangles to up-triangles and down-triangles to down-triangles, which is the only way to superimpose two polyiamonds that are congruent under pure translation in the actual Euclidean triangular tiling. Throughout this paper, the unqualified word "translation" means a parity-preserving lattice shift.

Fixed n-iamond. A polyiamond with exactly n cells. Two n-cell polyiamonds are considered distinct if they cannot be superimposed by a parity-preserving lattice shift alone. For $n = 1$ there are two distinct fixed 1-iamonds, the up-pointing triangle at $(0, 0)$ and the down-pointing triangle at $(0, 1)$. For $n \geq 2$ every fixed n-iamond already contains cells of both parities. We write $F(n)$ for the set of all fixed n-iamonds in canonical form and $|F(n)|$ for its cardinality; $|F(n)| = A001420(n)$ for every $n \geq 1$, matching the OEIS values $2, 3, 6, 14, 36, 94, 250, 675, 1838$ for $n = 1$ through 9.

Container. A connected polyiamond C is a container for $F(n)$ if, for every $P \in F(n)$, there exists a parity-preserving translation (dr, dc) such that $P + (dr, dc) \subseteq C$.

$\boldsymbol{a(n)}$**.** The minimum $|C|$ over all containers C for $F(n)$.

---

3. Computational Methodology

3.1. Main-window SAT solver

The main solver uses CaDiCaL 1.5.3 via PySAT with a top-down descent strategy. For each candidate target cell count k, the SAT formula encodes cell variables $x(r, c) \in \{0, 1\}$ for each cell of the search rectangle R_n with an exact-k cardinality constraint via the PySAT totalizer primitive, placement variables $y(i, t)$ for each fixed piece P_i and parity-preserving shift t with the usual piece-cell implication clauses and one at-least-one-placement coverage clause per piece, and connectivity via CEGAR (Counter-Example Guided Abstraction Refinement): after each

candidate model, a post-hoc BFS under the parity-dependent 3-neighbour adjacency checks component count, and disjunctive cuts are added for disconnected models until the returned model is a single connected component. An ITotalizer incremental cardinality wrapper is used so a single persistent solver instance serves the whole top-down descent.

Witness and UNSAT certificate. For each n, the search produces two artefacts: (i) a SAT witness at size $k = a(n)$, an explicit container satisfying every clause, and (ii) an UNSAT certificate at size $k = a(n) - 1$, proving by exhaustive SAT search that no container of that size exists inside R_n.

Search rectangle. The main solver's search domain for each n is R_n: 1×2 for $n = 1$ and $n \times n$ for $n \geq 2$. Every fixed n-iamond individually fits inside R_n, so placement enumeration is complete over all placements inside R_n. The SAT descent is exhaustive over all cell subsets of R_n of size k, for k ranging down from the initial upper bound.

3.2. Wider-window verification (Route C)

The main-window SAT descent proves the minimum inside R_n but, on its own, is silent on whether some smaller container exists outside R_n. To close that gap we re-run the search inside the strictly larger $(n+1) \times (n+1)$ rectangle W_n and emit a DRAT (Delete-Reverse-Asymmetric-Tautology) proof of UNSAT at size $a(n) - 1$. The DRAT proof is then independently verified by drat-trim (Wetzler, Heule, Hunt 2014, the de facto standard UNSAT proof checker), which produces verdict `s DERIVATION` when every lemma in the DRAT trace is a sound inference from the input formula.

The wider-window pipeline differs from the main-window solver on every axis: SAT backend (cadical-shim, a PySAT-Glucose-4.2 wrapper that emits DRAT in cadical's format, vs. CaDiCaL 1.5.3); search window ($W_n = (n+1) \times (n+1)$ vs. R_n); symmetry breaking (translation breaker pinning the bounding box vs. none); UNSAT certificate (DRAT proof vs. CaDiCaL's implicit verdict); and verifier (drat-trim built from the project's MSVC patch vs. none). For each $n \in \{1, ..., 9\}$ the wider-window pipeline returns the same minimum inside W_n as the main solver returned inside R_n, with verdict `s DERIVATION`.

3.3. Piece enumeration

Fixed n-iamonds are enumerated by the Cython-accelerated `polyform_enum` library operating on the triangular lattice, patched so that the $n = 1$ case returns both the up-triangle and the down-triangle. The resulting piece counts $2, 3, 6, 14, 36, 94, 250, 675, 1838$ for $n = 1$ through 9 match OEIS A001420 exactly.

3.4. Verification

Every reported term is cross-checked by three independent stacks with disjoint code paths from the main solver: verifier 1 (geometric containment, pure-Python BFS), verifier 2 (Glucose 4.2 with rooted spanning-arborescence connectivity and sequential-counter cardinality, re-deriving the UNSAT-at-$(k-1)$ inside R_n), and the wider-window verifier (cadical-shim with translation-symmetry breaking and DRAT emission, certified by drat-trim inside W_n). Across the four stacks (main solver + three verifiers), the only shared dependency at the code level is the parity rule $(r + c) \bmod 2$ and the 3-neighbour adjacency definition, both of which are reimplemented from scratch in each stack.

Verifier 1, verifier 2, and the wider-window verifier all pass for every $n \in \{1, ..., 9\}$. Verifier 2′s $n = 1$ through 9 sweep completed in approximately 741 s (dominated by the UNSAT-at-

$k = 30$ proof for $n = 9$ at about 649 s). The wider-window pipeline returns drat-trim verdict `s DERIVATION` for every $n \in \{1, ..., 9\}$ with cumulative wall time of approximately 70 s for the SAT solves and a similar amount for drat-trim verification.

3.5. Proof certification: prose trace and minimum unsatisfiable core

Beyond DRAT verification, the repository ships a prose-trace pipeline that (a) resolves the lower-bound instance with native cadical 3.0.0 (reliable DRAT flush on Windows; build recipe at `tools/cadical-support/`), (b) extracts the minimum unsatisfiable core via drat-trim's `-c` flag, and (c) renders a human-readable trace using the shared `sat_utils.prose_trace_container.ContainerTemplate`, constructed from a `ContainerDescriptor` derived from the project's own framework instance. Clause shapes are recognised without encoder retrofit: placement-implications (`-aux_p or cell_c`), piece-at-least-one, symmetry breakers, and CEGAR connectivity cuts.

The per-n certification statistics for $n \in \{1, ..., 5\}$ (`research/proof-trace.md`, regenerable via `python code/extract_proof.py --n N` after emitting DRATs):

n	$a(n)$	SAT vars	clauses	MUC	MUC frac	drat-trim
1	2	4	8	5	62.5%	VERIFIED
2	4	15	29	7	24.1%	VERIFIED
3	6	87	215	24	11.2%	VERIFIED
4	9	187	572	191	33.4%	VERIFIED
5	12	459	1883	816	43.3%	VERIFIED

Table 1: Prose-trace certification statistics. MUC is the minimum unsatisfiable core extracted by `drat-trim -c` in backward mode; the MUC fraction bounds the intrinsic complexity of the non-existence proof (at most MUC clauses are consumed to refute every $k = a(n) - 1$ candidate). All five proofs carry drat-trim's strongest verdict `s VERIFIED` (full refutation: every learned lemma is sound and the empty clause is derivable from the input formula), obtained by running cadical with `--no-binary` to force emission of an ASCII DRAT including the explicit empty-clause derivation step. Cadical's default binary DRAT format sometimes elides this final step for larger instances, degrading the verdict to `s DERIVATION` under the same checker; the ASCII flag makes the certification level uniform across all n.

The trace itself lists forced unit-propagation assignments in container vocabulary rather than opaque variable numbers – at $n = 3$, step 4 reads *"from placement-implication (piece 3 -> cell (1, 0)), force cell (1, 0) is occupied"*. Readers may reproduce the trace + MUC + implication DAG for any $n \in \{1, ..., 9\}$ via a single command after building the native cadical binary; see Reproducibility.

---

4. Proved Values

n	$a(n)$	$\lvert F(n)\rvert$ (A001420)	Main bbox	Main solver (s)	Wider-window (s)	Wider verdict
1	2	2	1×2	0.000	0.0	`s DERIVATION`
2	4	3	2×2	0.000	0.0	`s DERIVATION`
3	6	6	2×3	0.000	0.0	`s DERIVATION`
4	9	14	3×4	0.000	0.0	`s DERIVATION`
5	12	36	4×5	0.000	0.0	`s DERIVATION`
6	17	94	4×6	0.016	0.2	`s DERIVATION`
7	22	250	4×7	0.079	0.6	`s DERIVATION`
8	27	675	5×8	0.840	9.0	`s DERIVATION`
9	31	1838	6×9	4.911	58.7	`s DERIVATION`

Table 2: Proved values of $a(n)$ for $n = 1, ..., 9$. Each value is proved by the main solver's (witness, UNSAT-at-$(k-1)$) pair inside R_n and additionally certified by a drat-trim VERIFIED DRAT UNSAT proof inside $W_n = (n+1) \times (n+1)$.

Every row's main bbox has row span and column span at most n (with the 1×2 convention at $n = 1$), so every main-solver optimum fits inside R_n; the wider-window verifier additionally certifies that no smaller container exists inside the strictly larger W_n. Solver times below 1 ms are reported as 0.000 s.

---

5. Empirical Analysis and Conjectures

5.1. Growth rate

The first differences of $a(n)$ are $2, 2, 3, 3, 5, 5, 5, 4$ – non-monotone, with a plateau of three 5s at $n = 6, 7, 8$ followed by a drop to 4 at $n = 9$. The second differences are $0, 1, 0, 2, 0, 0, -1$, which is not constant, so $a(n)$ does not agree with any quadratic polynomial on these 9 terms. The ratio $a(n)/n^2$ drifts monotonically downward through $n = 9$: $2.000, 1.000, 0.667, 0.563, 0.480, 0.472, 0.449, 0.422, 0.383$. The sequence appears to grow asymptotically like $\Theta(n^2)$.

5.2. Cap structure conjecture

The optimal container for $n = 5$ has row span $4 = \lfloor \frac{5}{2} \rfloor + 2$, one row beyond the maximum row span of any single fixed 5-iamond (which is $\lfloor \frac{5}{2} \rfloor + 1 = 3$ by the parity-constrained vertical chain bound). The extra row is occupied by a single "cap" cell that is the unique witness location for some apex-up 5-iamond piece; without it, the container would need an extra cell elsewhere. The same phenomenon recurs at $n = 9$ with two cap cells in the top row and one in the bottom row, again saving cells compared to the $\lfloor \frac{n}{2} \rfloor + 1$ row-bounded construction. No cap row appears at $n = 2, 3, 4, 6, 7, 8$.

The cap rows so far appear at $n = 5$ and $n = 9$ – both $n \equiv 1 \pmod 4$. With only two data points the pattern is conjectural, but if it persists then $n = 13$ should show a cap saving of

approximately three cells. The cap-row phenomenon is a real structural feature of the minimum container, not a numerical accident.

5.3. Falsification of the A024206 conjecture

A prior-art pass noted that an earlier (buggy) solver's prefix coincided with a shift of OEIS A024206 and recorded a provisional "FOUND (extending A024206)" verdict. That earlier solver used a retired `use_shape` pruning heuristic and had a triangular parity bug. After repairing both bugs and the $A001420(1)$ bug, the re-verified values $2, 4, 6, 9, 12, 17, 22, 27, 31$ were compared term-by-term against every shift $A024206(n + k)$ for $k \in \{0, ..., 5\}$ during the conjecture-search step. No shift produces a prefix matching all nine proved terms, and the disagreement begins at or before $n = 6$ for every shift tried. The prior-art verdict has accordingly been revised from "FOUND (extending)" to "CLEAR".

5.4. No closed-form conjecture

The conjecture-search step tested seven candidate formulas across five distinct categories: minimal-degree rational polynomial fit (no fit); quadratic through three anchor points (predicts wrong values at intermediate n); the closed form $a(n) = \frac{n(n+1)}{2}$ (fails at $n = 2$); the density heuristic $a(n) = \left\lceil 3\frac{n^2}{8} \right\rceil$ (fails at $n = 2$); Berlekamp-Massey linear recurrences of order up to 5 (no fit); A024206 shifts (rejected above); and a power-law fit (no clean fit). No formula matches all nine proved terms; the outcome is "no conjecture found".

Beyond simple curve-fitting, four standard structural lower-bound techniques were attempted in the project's open-problem-A395422 research thread and are documented as inapplicable to this sequence: pairwise minimum-cell-superform overlap (the technique behind OEIS A327094 and Muniz's pentomino MCS), Barequet and Ben-Shachar 2022 inflation rearrangement, fractional-cover LP relaxation with weighted-mass duality, and local-exchange / notch-shift moves on container shapes. The first three are bounded by linear-in-n quantities and cannot match the $\Theta(n^2)$ growth of $a(n)$ for $n \geq 4$; the local-exchange approach yields the right reshape template at $n = 5$ but does not generalise without further case analysis.

---

6. Discussion and Open Problems

> **Open Problem 1 (unrestricted Assumption (S)).** Prove, or disprove by explicit counterexample, that for every $n \geq 1$ there exists a minimum container for $F(n)$ whose bounding box is contained in some bounded rectangle $(n + k(n)) \times (n + k(n))$ with $k(n)$ bounded uniformly in n. A proof for any specific bound $k(n)$ – in particular $k(n) = 1$, the wider-window case discharged for $n = 1, ..., 9$ in this paper – would extend Theorem 1 to all n by the same wider-window pipeline.

For $n = 1, ..., 9$ the bounding boxes of the main-window optima are $1 \times 2, 2 \times 2, 2 \times 3, 3 \times 4, 4 \times 5, 4 \times 6, 4 \times 7, 5 \times 8, 6 \times 9$ – always inside R_n. The wider-window certificate strengthens this to: no smaller container exists inside $W_n = (n + 1) \times (n + 1)$ either. The unbounded n case remains open.

In the project's open-problem-A395422 research thread, four classical structural attacks on (S) were attempted and ruled out for this sequence's growth regime:

- **Pairwise minimum-cell-superform overlap.** The pairwise lower bound on $|C|$ is at most $|P| + |Q| - \text{overlap} \leq 2n$; but $a(n)$ grows like $\Theta(n^2)$, so the technique cannot scale beyond $n = 4$.
- **Isoperimetric / inflation rearrangement** (Barequet and Ben-Shachar, 2022, "Minimum-Perimeter Lattice Animals and the Constant-Isomer Conjecture", *Electronic Journal of Combinatorics* 29(3), P3.45). Their inflation theorem characterises minimum-*perimeter* animals, not minimum-cell containers; the second premise of their main theorem fails for polyiamonds (the "jaggy" failure mode), and their patched definition uses vertex-adjacency rather than edge-adjacency, incompatible with our edge-connected container.
- **LP duality / weighted-mass cover bounds.** The natural LP relaxation has integrality gap that grows linearly in n (LP optimum saturates near 3–4 cells while $a(n)$ grows quadratically); the constraint matrix is not totally unimodular.
- **Local exchange / notch-shift on container shapes.** A specific reshape – delete a "bridge" row connecting an off-axis cap cell to the bulk, then re-anchor the cap directly above the bulk – has been verified at $n = 5$ to take r-span $= 5$ down to r-span $= 4$ without increasing $|C|$. The single-cell move set is too narrow to handle multi-cap configurations ($n = 9$ onwards).

Together these results suggest that any structural proof of (S) will require techniques heavier than single-piece overlap or fractional-cover relaxation – a transfer-matrix decomposition or a more elaborate move calculus on container shapes are the most promising remaining angles.

1. **Extending the computation to $n = 10, 11, 12$.** The wider-window pipeline shipped in 2026-04-20 makes per-n verification practical: at $n = 9$ the wider-window solve takes about 59 s and drat-trim verification takes another 40 s, single-threaded. At $n = 10$ the main solver takes about 154 s for the $n \times n$ window and the wider-window solve at 11×11 is estimated at roughly 20 min. $n = 11$ is reachable on dedicated compute at an estimated 48-hour budget.

2. **No closed-form conjecture.** The nine known terms do not match any polynomial, linear recurrence, or simple known-sequence template we have tested. A closed form – if one exists – likely involves floor/ceiling or modular case splits (consistent with the cap-row conjecture above).

3. **Relationship to known polyiamond sequences.** The piece count $|F(n)| = A001420(n)$ grows much faster than $a(n)$ (1838 vs 31 at $n = 9$), confirming that containers overlap aggressively. The free polyiamond count A000577 and the one-sided polyiamond count A006534 are companions of A001420 in the same combinatorial family, but neither appears as a subsequence or transformation of our sequence.

4. **Analogs on other grids.** The hex grid analog gives $a(n) = \frac{n(n+1)}{2}$ (triangular numbers, OEIS A000217). The square grid analog for fixed polyominoes is unexplored in the recent literature; the square grid free variant is OEIS A327094. The present triangular-grid fixed variant has, to our knowledge, no prior systematic computation.

---

7. Reproducibility

All code and data are in the project repository. The main solver entry point, the two verifiers, the wider-window pipeline, the prose-trace driver, and all JSON outputs are listed in the Markdown source of this paper. Dependencies: Python 3.12+, python-sat (PySAT with CaDiCaL 1.5.3 and

Glucose 4.2), `polyform_enum` (Cython polyiamond enumerator, patched for the $A001420(1) = 2$ case), `sat_utils` (shared-library cardinality, connectivity, placement-enumeration, wider-window wrappers, and prose-trace toolchain including `ContainerTemplate`), drat-trim (built from the project's MSVC patch at `tools/drat-trim-support/drat-trim-msvc.patch` on Windows; standard build on Linux), and native cadical 3.0.0 (required for the prose-trace pipeline on Windows, where the PySAT binding does not reliably flush DRAT; build recipe at `tools/cadical-support/`).

Hardware for reported timings. AMD Ryzen 5 5600, Windows 11, single-threaded.

8. Acknowledgements

The author thanks the OEIS Foundation for maintaining the database of integer sequences, and the developers of PySAT, CaDiCaL, Glucose, and drat-trim for the SAT-solving and proof-checking infrastructure that makes exhaustive proofs of this kind practical.

9. Bibliography

1. A001420. "Number of fixed 2-dimensional triangular-celled animals with n cells (fixed polyiamonds; rotations and reflections are distinct)." OEIS Foundation, https://oeis.org/A001420
2. A000577. "Number of free polyiamonds with n cells (rotations and reflections identified)." OEIS Foundation, https://oeis.org/A000577
3. A006534. "Number of one-sided polyiamonds with n cells (rotations identified, reflections distinct)." OEIS Foundation, https://oeis.org/A006534
4. A000217. "Triangular numbers." OEIS Foundation, https://oeis.org/A000217
5. A024206. "Quarter-squares minus 1." OEIS Foundation, https://oeis.org/A024206
6. A327094. "Smallest polyomino containing all free n-ominoes." OEIS Foundation, https://oeis.org/A327094
7. Biere, A., Fazekas, K., Fleury, M., Heisinger, M. "CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020." Proc. SAT Competition 2020.
8. Ignatiev, A., Morgado, A., Marques-Silva, J. "PySAT: A Python Toolkit for Prototyping with SAT Oracles." Proc. SAT 2018, pp. 428–437.
9. Audemard, G., Simon, L. "Predicting learnt clauses quality in modern SAT solvers." Proc. IJCAI 2009, pp. 399–404. (Glucose)
10. Wetzler, N., Heule, M. J. H., Hunt, W. A. "DRAT-trim: Efficient Checking and Trimming Using Expressive Clausal Proofs." Proc. SAT 2014, pp. 422–429.
11. Barequet, G., Ben-Shachar, G. "Minimum-Perimeter Lattice Animals and the Constant-Isomer Conjecture." *Electronic Journal of Combinatorics* 29(3), P3.45, 2022. https://doi.org/10.37236/10770